Lunch Picker Technical Documentation

Structured functional and technical specification for a web-based group lunch coordination and voting platform

1. Description of the Project

The **Lunch Picker** is a web-based application designed to enable groups of users (such as office teams, project squads, or social clubs) to seamlessly coordinate daily or weekly lunch choices. The system provides a structured, automated workflow for proposing restaurants, curating tailored menus, casting votes, and automatically selecting venues through configurable decision algorithms (randomization, consensus, or weighted preferences).

All primary interactions are conducted through a responsive Django web interface, backed by a clean RESTful API designed for seamless future mobile and client integrations.

2. Goal of the Project

Create a reliable, extensible, and high-performance platform that:

- Allows authorized users to create, configure, and manage teams and memberships.
- Supports comprehensive registration and cataloging of restaurants and menu items.
- Enables scheduling of "lunch sessions" where participants vote on menus or venues.
- Provides automatic venue selection based on rules such as random draw, majority vote, or weighted preference.
- Stores historical data for analytics, reporting, and discovering recurring team dining patterns.
- Enforces strict security, data integrity, and a maintainable code base aligned with Django best practices.

3. Technologies Used

- **Backend:** Python 3.11, Django 5.0, Django REST Framework (DRF)
- **Database:** PostgreSQL 15 (with optional SQLite support for local development)
- **Frontend:** HTML5, CSS3 (Tailwind CSS or Bootstrap 5), JavaScript (ES6+)
- **Authentication:** Django built-in auth system, DRF SimpleJWT, with optional OAuth2 (Google/Microsoft)
- **Deployment:** Docker, Docker Compose, Gunicorn, Nginx, CI/CD via GitHub Actions
- **Testing & Quality:** pytest-django, factory-boy, coverage.py

4. System Functionalities

4.1 Administrator Actions

- Manage all system users, teams, and global application settings.
- Create, update, edit, or delete any restaurant and associated menu entry across the platform.
- Define system-wide selection algorithms, weighting schemes, and voting rules.
- Access comprehensive system analytics and usage metrics dashboards.

4.2 Team Lead (Group Owner) Actions

- Create and configure new teams, issue member invitation links, and assign permissions/roles.
- Schedule one-time or recurring lunch sessions.
- Propose restaurants or specific menu items for upcoming lunch sessions.
- Configure session-specific voting parameters (e.g., voting deadlines, anonymity).

4.3 Regular User Actions

- Join existing teams via dedicated invitation links.
- View upcoming lunch sessions and explore proposed dining choices.
- Cast votes using flexible schemes (single-choice, multiple-choice, or rating scale).
- Receive automated notifications regarding the final selected venue.
- Browse personal and team lunch history, with filtering capabilities by date range or venue.

5. Core Data Models / Entities

ENTITY	FIELDS & RELATIONSHIPS
User	`username`, `email`, `first_name`, `last_name`, `is_active`, `is_staff`, `date_joined`
Team	`name`, `description`, `created_by` (FK User), `members` (M2M User via Membership), `created_at`
Restaurant	`name`, `address`, `cuisine_type`, `website_url`, `average_price`, `rating`, `created_by` (FK User)
MenuItem	`restaurant` (FK Restaurant), `name`, `description`, `price`, `is_vegetarian`, `is_vegan`, `is_gluten_`
LunchSession	`team` (FK Team), `title`, `start_time`, `end_time`, `is_recurring`, `recurrence_rule`, `selection_algorithm`, `created_by` (FK User), `created_at`
SessionProposal	`session` (FK LunchSession), `restaurant` (FK Restaurant), `menu_items` (M2M MenuItem), `proposer` (User), `created_at`
Vote	`session` (FK LunchSession), `voter` (FK User), `proposal` (FK SessionProposal), `choice` (like, rating 1- `voted_at`

ENTITY	FIELDS & RELATIONSHIPS
SelectionResult	`session` (FK LunchSession), `selected_proposal` (FK SessionProposal), `method_used` (enum), `execu`
LunchHistory	Aggregated read-only view over `SelectionResult` entries

6. API Endpoints & Architecture

The system provides a RESTful architecture powered by Django REST Framework (DRF). All endpoints utilize class-based ViewSets or Generics, protected by granular permission classes (`IsAuthenticated` , `IsAdminUser` , custom `IsTeamOwner` , `IsTeamMember`). Filtering, searching, and pagination are enabled globally.

ENDPOINT PATTERN	HTTP METHODS	DESCRIPTION & ACCESS CONTROL
`/api/auth/`	POST	Token obtain and refresh endpoints (DRF SimpleJWT).
`/api/users/`	GET PUT PATCH	Manage user profiles. Admin can list all users; users manage own profiles.
`/api/teams/`	GET POST PUT DELETE	CRUD operations for teams. Write operations restricted to Team Owner/Admin.
`/api/teams/{id}/members/`	GET POST DELETE	List team members, issue invites, or remove users.
`/api/restaurants/`	GET POST PUT DELETE	Global restaurant catalog. Admin write access; general read access.
`/api/restaurants/{id}/menu-items/`	GET POST PUT DELETE	Manage specific dishes associated with a restaurant catalog item.
`/api/sessions/`	GET POST PUT	Create and list lunch sessions. Team leads hold creation privileges.
`/api/sessions/{id}/proposals/`	GET POST	Submit restaurant/menu proposals for an active session.
`/api/sessions/{id}/votes/`	GET POST PUT	Cast or modify votes for proposals within an active session.
	GET	

ENDPOINT PATTERN	HTTP METHODS	DESCRIPTION & ACCESS CONTROL
`/api/sessions/{id}/result/`		Retrieve final venue selection result after session closure.
`/api/analytics/team/{id}/`	GET	Summary analytics for teams with date range filtering.

7. Security & Access Control

- **Transport Security:** Enforce strict HTTPS at Nginx / load balancer level with SSL termination.
- **Authentication & CSRF:** CSRF protection on HTML UI views; DRF `TokenAuthentication` (JWT) for API interactions.
- **Password Storage:** Secure password hashing via Argon2 (Django's default algorithm).
- **Granular Role-Based Permissions:** Role hierarchies separating Global Admin, Team Owner, Team Member, and Read-Only Guest access.
- **Injection & XSS Prevention:** Input sanitization via Django ORM parameterized queries; auto-escaping enabled in HTML templates.
- **Rate Limiting:** Enforce rate limiting on sensitive API endpoints (e.g., maximum 5 vote submissions per minute per user using `django-ratelimit`).
- **Audit Logging:** Centralized logging for sensitive operations (session creation, vote changes, selection algorithm triggers).

8. Business Logic & Validation Rules

- **Model Validation:** Enforce constraints via `clean()` methods (e.g., session `end_time` must be after `start_time`; votes must correspond to proposals within the exact session).
- **Serializer Constraints:** Validate payload integrity in DRF serializers (e.g., enforce unique restaurant names per team scope).
- **Form Validation:** Dual-layer client-side and server-side validation mirroring API rules for immediate feedback.

9. Module Structure (Django Project Layout)

```
lunchpicker/                # Project root
│
├─ lunchpicker/             # Settings, URLs, WSGI/ASGI configuration
│   ├─ __init__.py
│   ├─ settings.py
│   ├─ urls.py
│   └─ wsgi.py
│
├─ accounts/                # User model extensions, authentication, profiles
│   ├─ models.py
│   ├─ serializers.py
│   ├─ views.py
│   └─ urls.py
│
├─ teams/                   # Team management, membership handling
│   ├─ models.py
│   ├─ serializers.py
│   ├─ views.py
│   └─ urls.py
│
├─ restaurants/             # Restaurant catalog and menu items
│   ├─ models.py
│   ├─ serializers.py
│   ├─ views.py
│   └─ urls.py
│
├─ sessions/                # Lunch session workflow, proposals, and voting
│   ├─ models.py
│   ├─ serializers.py
│   ├─ views.py
│   └─ urls.py
│
├─ analytics/               # Reporting, stats, and historical aggregate views
│   ├─ models.py (read-only views)
│   ├─ serializers.py
│   ├─ views.py
│   └─ urls.py
│
├─ core/                    # Shared utilities, permissions, custom validators
│   ├─ permissions.py
│   ├─ validators.py
│   └─ utils.py
│
├─ tests/                   # pytest test suites organized by application module
│   └─ …
│
├─ Dockerfile
├─ docker-compose.yml
└─ requirements.txt
```

10. Future Expansion Opportunities

1. **Third-Party Integrations:** OAuth2 logins (Google, Microsoft), automated calendar sync for lunch slots, and Slack/Teams bots for automated voting reminders.

2. **Recommendation Engine:** Machine learning algorithms suggesting restaurants based on historical preferences, dietary constraints, and group trends.
3. **Mobile Applications:** Native iOS/Android clients consuming the DRF API, featuring real-time push notifications.
4. **Advanced Voting Schemes:** Support for ranked-choice voting, preference-weighted algorithms, and split-bill calculations.
5. **Payment Integration:** Direct order cost splitting integrated via Stripe or PayPal APIs.
6. **Multi-tenant SaaS Offering:** Complete organizational tenant isolation, custom branding, and subscription management.
7. **Interactive Analytics Dashboard:** Rich front-end data visualizations (Chart.js / D3.js) tracking cuisine trends, spending, and participation.